

Instructions:

- Answer the questions in the spaces provided on the question sheets.
- Make sure to write your name and ERP on each page.
- Extra sheets may be used for rough work, but make sure to write your name and ERP on them. Answers on extra sheets will not be graded.
- Understanding the questions is part of the exam, please do not ask for clarifications during the exam.
- Make sure you have all 20 pages (printed on 10 two sided sheets) of the exam before you begin.

Name: _____

Solution

ERP: _____

Parts:	Part I (Tracing)	Part II (Short Questions)	Part III (Implementation)	Total
Marks:	35	35	30	100
Score:				

Part I (Tracing)

Question 1. Stacks and Queues 12 marks

- 3 marks** (a) Consider the following code snippet from your **"Dungeon Escape"** game. A custom queue is used to manage the challenges faced by the player. Trace the state of the queue after each operation.

```
CustomQueue challenges;
challenges.enqueue("Fight a dragon!");
challenges.enqueue("Solve a riddle!");
challenges.enqueue("Avoid a trap!");
challenges.dequeue();
challenges.enqueue("Defeat a skeleton!");
challenges.dequeue();
```

Questions

1. What is the state of the queue after line 3 (i.e., after all challenges are enqueued)?
2. What is the state of the queue after line 4 (i.e., after the first dequeue)?
3. What is the state of the queue after line 5 (i.e., after enqueueing "Defeat a skeleton!")?
4. What is the final state of the queue after line 6 (i.e., after the second dequeue)?

Answer Template

1. After line 3: "Fight a dragon!", "Solve a riddle!", "Avoid a trap!"
2. After line 4: "Solve a riddle!", "Avoid a trap!"
3. After line 5: "Solve a riddle!", "Avoid a trap!", "Defeat a skeleton!"
4. After line 6: "Avoid a trap!", "Defeat a skeleton!"

- 3 marks** (b) Consider the following code snippet from your **"Dungeon Escape"** game. A custom stack is used to track the player's path through the dungeon. Trace the state of the stack after each operation.

```
CustomStack path;
path.push("Room 1: Entrance");
path.push("Room 2: Dark Corridor");
path.push("Room 3: Treasure Room");
path.pop();
path.push("Room 4: Exit");
path.pop();
path.pop();
```

Questions

1. What is the state of the stack after line 3 (i.e., after all rooms are pushed)?
2. What is the state of the stack after line 4 (i.e., after the first pop)?
3. What is the state of the stack after line 5 (i.e., after pushing "Room 4: Exit")?
4. What is the final state of the stack after line 7 (i.e., after all pops)?

Answer Template

1. After line 3: "Room 1: Entrance", "Room 2: Dark Corridor", "Room 3: Treasure Room"
2. After line 4: "Room 1: Entrance", "Room 2: Dark Corridor"
3. After line 5: "Room 1: Entrance", "Room 2: Dark Corridor", "Room 4: Exit"
4. After line 7: "Room 1: Entrance"

- 3 marks (c) Consider the following code snippet from the "Dungeon Escape" game. A custom queue is used to manage the challenges faced by the player. Determine the output of the code.

```
CustomQueue challenges;  
challenges.enqueue("Fight a dragon!");  
challenges.enqueue("Solve a riddle!");  
challenges.enqueue("Avoid a trap!");  
cout << challenges.front() << endl;  
challenges.dequeue();  
cout << challenges.front() << endl;  
challenges.enqueue("Defeat a skeleton!");  
challenges.dequeue();  
cout << challenges.front() << endl;
```

Answer Template

The output of the code is:

1. Fight a dragon!
2. Solve a riddle!
3. Avoid a trap!

- 3 marks (d) Consider the following code snippet from the "Dungeon Escape" game. A custom stack is used to track the player's path through the dungeon. Determine the output of the code.

```
CustomStack path;  
path.push("Room 1: Entrance");  
path.push("Room 2: Dark Corridor");  
path.push("Room 3: Treasure Room");  
cout << path.top() << endl;  
path.pop();  
cout << path.top() << endl;  
path.push("Room 4: Exit");  
path.pop();  
cout << path.top() << endl;
```

Answer Template

The output of the code is:

1. Room 3: Treasure Room
2. Room 2: Dark Corridor
3. Room 2: Dark Corridor

Question 2. Constructor Types 10 marks

```
#include <iostream>
using namespace std;

class Test {
public:
    Test() { cout << "Default Constructor" << endl; }
    Test(const Test& other) { cout << "Copy Constructor" << endl; }
    Test& operator=(const Test& other) {
        cout << "Assignment Operator" << endl;
        return *this;
    }
};

int main() {
    Test* ptr1 = new Test();
    Test obj1 = *ptr1;
    Test obj2;
    obj2 = obj1;
    delete ptr1;
    return 0;
}
```

a) What will be the output of the code given above? 4 marks

Default Constructor
Copy Constructor
Default Constructor
Assignment Operator

```
class Value {
public:
    Value() { std::cout << "Default\n"; }
    Value(const Value& v) { std::cout << "Copy\n"; }
};

void process(Value v) { // ... }

int main() {
    Value a;
    process(a);
    return 0;
}
```

b) What will be the output of the code given above? 2 marks

Default
Copy

- c) Explain the sequence of constructor calls. How would the output change if the function **process** took a Value& v as an argument? 4 marks

The default constructor is called when object a is created and then the copy constructor is called when it is passed to the process function as a value.

If the function took the object's reference, then the copy constructor would not be called.

Output: Default

Question 3. Static and Const 2 marks

```
class Demo {
public:
    static void func() const
    {
        cout << "I am func";
    }
};
Int main() {
    Demo::func();
}
```

- a) What is the problem with the code given above? Explain 2 marks

The issue is that static functions cannot be const because const applies to instance methods by preventing modifications to member variables, while static methods do not belong to any instance.

Question 4. Exceptions 3 marks

```
class Test {
public:
    Test() {
        cout << "Constructor called" << endl;
        throw runtime_error("Constructor exception");
    }
    ~Test() {
        cout << "Destructor called" << endl;
    }
};
```

```
int main() {
    try {
        Test t;
    } catch (const exception& e) {
        cout << "Caught: " << e.what() << endl;
    }
    return 0;
}
```

- a) What will be the output of the code given above? Explain whether the destructor of the Test class be called or not after the exception is thrown in the constructor and why. 3 marks

Output:

Constructor called

Caught: Constructor exception

Since the exception is thrown inside the constructor, the object is not fully created, and its destructor is **not** called.

Question 5. Constructors and Destructors 8 marks

```
#include <iostream>
using namespace std;
class A {
    inline static int count = 0;
public:
    A() {
        count++;
        cout << "A(" << count << ")" << endl;
    }
    ~A() {
        cout << "~A(" << count << ")" << endl;
        count--;
    }
};

class B {
    A a;
public:
    B() {
        cout << "B()" << endl;
        foo();
    }
    ~B() {
        cout << "~B()" << endl;
    }
    virtual void foo() {
        A a;
    }
};
```

```
class C : public B {
public:
    C() {
        cout << "C()" << endl;
        foo();
    }
    ~C() {
        cout << "~C()" << endl;
    }

    void foo() {
        A a;
        B::foo();
    }
};

int main()
{
    B *b = new C();
    b->foo();
    delete b;
}
```

a) What will be the output of the code given above? 4 marks

```
A(1)
B()
A(2)
~A(2)
C()
A(2)
A(3)
~A(3)
~A(2)
A(2)
A(3)
~A(3)
~A(2)
~B()
~A(1)
```

b) Is any constructor/destructor call being missed? If yes, how can we prevent it from being missed? 4 marks

The destructor class C is being missed because the pointer of the base class B is being used to delete the object. We can prevent it from being missed by making the destructor of class B virtual due to which the overridden destructor in the derived class C will be called because of dynamic binding.

Part II (Short Questions)**Question 6.** Encapsulation 3 marks

A class **V** contains a private integer variable named **value**, along with functions to give that variable a new value:

```
class V {  
public:  
    void setValue(int value);  
...  
private:  
    int value;  
};
```

Your colleague is trying to implement this function and has this code in file V.cpp.

```
// set value  
void V::setValue(int value) {  
    value = value;  
}
```

Something's wrong – the variable **value** in the object isn't getting updated when this function is called.

a) What change could you make to the body of the function only that would fix the problem? 3 marks

The problem is that the argument **value** of the function **setValue** has the same name as the attribute **value** of class **V** which makes the function argument shadow the class attribute. Hence, effectively the function argument is updating itself instead of the class attribute.

To fix this problem use **this** pointer to access the attribute of the class. Fixed code: `this->value = value;`

Question 7. Function Overloading 6 marks

Suppose the following functions have been declared

```
void exam(int n); // 1  
void exam(double x, int n); // 2  
void exam(int n, double x); // 3  
void exam(double x, double y); // 4
```

For each of the following function calls, indicate which overloaded function (1, 2, 3, or 4) is actually called. If the call cannot be resolved to a unique function, describe why this can't be done.

a. `exam(3.14, 1);` 2 marks

3.14 is a double, and 1 is an int.

Function 2 is the best match because it exactly matches the argument types.

b. `exam(3.14, 1.0);` 2 marks

3.14 is a double, and 1.0 is a double.

Function 4 is the best match since both arguments are double.

c. `exam(3, 1);` 2 marks

3 is an int, and 1 is an int.

Both Function 2 and Function 3 require one implicit conversion (either $3 \rightarrow \text{double}$ or $1 \rightarrow \text{double}$). Since both conversions are equally valid, the compiler cannot determine the best match.

Compilation Error: Ambiguous Call

Question 8. Inheritance **11 marks**

Consider the following C++ code:

```
#include <iostream>
using namespace std;
class A {
protected:
    int x;
public:
    A(int val) : x(val) {}
    void display() {
        cout << "Class A: x = " << x << endl;
    }
};

class B : public A {
public:
    B(int val) : A(val) {}
    void display() {
        cout << "Class B: x = " << x * 2 << endl;
    }
};
```

```
class C : public B {
public:
    C(int val) : B(val) {}
    void display() {
        cout << "Class C: x = " << x * 3 << endl;
    }
};

int main() {
    A* obj = new C(5);
    obj->display();
    return 0;
}
```

a) Explain the role of Protected access specifier in class A **1 mark**

In class A, the variable x is declared as protected, this means:

- x is not accessible from outside the class (e.g., in main()), unlike public members.
- x is accessible within derived classes (B and C).
- If x were private, even B and C wouldn't be able to access it directly, and a getter function would be required.

b) How does Protected affect classes B and C? **1 mark**

- Since B and C inherit from A, they can directly use x in their member functions.
- For example, display() in class B and C can manipulate x because x is protected:

```
void display() {
    cout << "Class B: x = " << x * 2 << endl; }
```

 Here, x is directly used inside B, which is only possible because it was declared as protected in A.

c) Predict the output of the program **1 mark**

Predicted Output:
Class A: x = 5

d) Explain why function display() is called from the Base class but not from the derived class? **2 marks**

- In main(), obj is a pointer of type A* but points to an object of class C.
- Since there is no virtual function, function overriding does not work through the base class pointer.
- Instead, function hiding occurs, meaning obj->display(); calls A::display() because display() in B and C are hidden from the base class perspective.
- If display() were declared as virtual in A, the overridden version in C would have been executed.

- e) How can we modify the code in main() (without changing the code of the classes) so that calling obj->display(); in main() invokes display() from class C and explain why it will work 3 marks

We can change the pointer type:

```
int main()
```

```
{
    C* obj = new C(5); // Change pointer type from A* to C* obj->display();
    return 0;
}
```

Now, the output will be: Class C: x = 15

Since obj is now explicitly a C* pointer, it directly calls C::display().

- f) How can we modify the code of the classes (without changing the code in main()) so that calling obj->display(); in main() invokes display() from class C and explain why it will work 3 marks

We can make the display function virtual:

```
Class A {
```

```
    ...
    virtual void display() { // Make display function virtual
    ...};
```

Now, the output will be: Class C: x = 15

The call using obj which is a pointer of class A will now be dynamically bound to the object of class C

Question 9. Polymorphism 15 marks

Consider the following C++ code:

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() {
        cout << "Animal Sound" << endl;
    }

    void speak() {
        cout << "Animal Speaking" << endl;
        sound();
    }
};

class Mammal : public Animal {
public:
    void sound() override {
        cout << "Mammal Sound" << endl;
        Animal::sound();
    }
};
```

```
class Dog : public Mammal {
public:
    void sound() override {
        cout << "Dog Bark" << endl;
    }

    void wagTail() {
        cout << "Dog Wagging Tail" << endl;
    }
};

class Cat : public Animal {
public:
    void purr() {
        cout << "Cat Purring" << endl;
    }
};

int main() {
    Animal* pet1 = new Dog();
    Animal* pet2 = new Mammal();
    Animal* pet3 = new Animal();
    void* pet4 = new Mammal();
    Mammal* pet5 = new Dog();
    void* pet6 = new Cat();
}
```

For each of the statements below, determine the output or whether it results in a compiler error or runtime

Name/Erp: _____

error. If the output consists of multiple lines, separate them using slashes (/). 1 mark each

Statement	Output
a) ((Mammal*)pet4)->sound();	Mammal Sound / Animal Sound
b) pet5->sound();	Dog Bark
c) ((Cat*)pet6)->sound();	Animal Sound
d) pet3->speak();	Animal Speaking / Animal Sound
e) ((Mammal*)pet4)->speak();	Animal Speaking / Mammal Sound / Animal Sound
f) pet5->speak();	Animal Speaking / Dog Bark
g) ((Cat*)pet6)->speak();	Animal Speaking / Animal Sound
h) ((Mammal*)pet1)->wagTail();	wagTail() is not defined in Mammal, so compiler error
i) ((Cat*)pet6)->purr();	Cat Purring
j) ((Dog*)pet4)->sound();	Invalid cast, leading to undefined behavior (likely crash at runtime)
k) ((Animal*)pet6)->speak();	Animal Speaking / Animal Sound
l) ((Animal*)pet4)->sound();	Mammal Sound / Animal Sound
m) ((Dog*)pet6)->wagTail();	Invalid cast, leading to undefined behavior (likely crash at runtime)
n) ((Dog*)pet3)->wagTail();	Invalid cast, leading to undefined behavior (likely crash at runtime)
o) ((Dog*)pet5)->wagTail();	Dog Wagging Tail

Part III (Implementation)**Question 10.** Encapsulation **10 marks**

Define a class called `NovelInfo` that stores information about a novel and how it is rated by readers (real numbers between 0.0 and 5.0). The class should have the following public methods:

- a. Constructor:
2 marks `NovelInfo(string title, string writer)` – Initializes a `NovelInfo` object with the given title and writer.
- b. Review Method:
2 marks `void addReview(double score)` – Records a review for the novel with the given rating. If the rating is not between 0.0 and 5.0, it should be ignored.
- c. Accessors:
 - 2 marks i. `string getTitle() const` – Returns the title of the novel.
 - 2 marks ii. `double getAverageScore() const` – Returns the average of all ratings (returns 0.0 if there are no reviews).
- d. String Representation:
 - 2 marks i. `string displayInfo() const` – Returns a string in the format:
"<title>, by <writer>, <score> (<count> reviews)"
 - The score should be truncated to one decimal place (not rounded).
 - If there is exactly one review, it should say "1 review" instead of "1 reviews".

Assume the following code is executed:

```
NovelInfo novel("1984", "George Orwell");
novel.addReview(4.7);
novel.addReview(5.0);
novel.addReview(4.9);
novel.addReview(4.9);
cout << novel.getAverageScore() << endl;
cout << novel.displayInfo() << endl;
```

After these calls:

- `novel.getAverageScore()` should return 4.8 (the average of the ratings).
- `novel.displayInfo()` should return: **"1984, by George Orwell, 4.8 (4 reviews)"**

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

class NovelInfo {
private:
    string title;
    string writer;
    vector<double> reviews;

public:
    NovelInfo(string title, string writer) : title(title),
                                             writer(writer) {}

    void addReview(double score) {
        if (score >= 0.0 && score <= 5.0) {
            reviews.push_back(score);
        }
    }

    string getTitle() const {
        return title;
    }
}
```

```
double getAverageScore() const {
    if (reviews.empty()) {
        return 0.0;
    }

    double sum = 0.0;
    for (double score : reviews) {
        sum += score;
    }

    return int((sum / reviews.size()) * 10) / 10.0;
}

string displayInfo() const {
    double avgScore = getAverageScore();
    int count = reviews.size();

    return title + ", by " + writer + ", " +
           to_string(avgScore).substr(0, 3) + " (" +
           to_string(count) + " review" +
           (count == 1 ? "" : "s") + ")";
}
};
```

Question 11. Queue and Stack ADTs 10 marks

- (a) Consider the given problem and complete the missing code.

The definition of the function `moveNthFront` that takes as a parameter a positive integer, `n`. The function moves the `n`th element of the queue to the front. The order of the remaining elements remains unchanged. For example, suppose:

queue = {5, 11, 34, 67, 43, 55} and `n = 3`

After a call to the function `moveNthFront`:

queue = {34, 5, 11, 67, 43, 55}

Implement the missing functions for the `Queue` class: `enqueue`, `dequeue`, and `moveNthFront`.

The rest of the code is provided below.

```
#include <iostream>

using namespace std;

// Node structure for the queue
struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(NULL) {}
};

// Queue class
class Queue {
private:
    Node* front;
    Node* rear;
    int size;
public:
    Queue() : front(NULL), rear(NULL), size(0) {}

    // TODO: Implement the enqueue function
    void enqueue(int val) {
```

```
        Node* newNode = new Node(val);

        if (rear == NULL)
        {
            front = rear = newNode;
        }
        else
        {
            rear->next = newNode;
            rear = newNode;
        }

        size++;
```

```
}
```

```
// TODO: Implement the dequeue function
```

```
1 mark int dequeue() {
```

```
    if (front == NULL)
    {
        cout << "Queue is empty!" << endl;
        return -1;
    }

    int val = front->data;

    Node* temp = front;

    front = front->next;

    if (front == NULL)
    {
        rear = NULL;
    }

    delete temp;
    size--;
    return val;
```

```
}
```

```
// Get front element without removing it
```

```
int front() {
    if (!front) return -1;
    return front->data;
}
```

```
// Check if queue is empty
```

```
bool isEmpty() {
    return front == NULL;
}
```

```
// Get the size of the queue
```

```
int getSize() {
```

```

        return size;
    }

    // TODO: Implement the moveNthFront function
2 marks void moveNthFront(int n) {

        if (n <= 0 || n > size) {
            cout << "Invalid position!" << endl;
            return;
        }

        if (n == 1) return; // Already at the front

        Node* prev = NULL;
        Node* curr = front;
        for (int i = 1; i < n; i++) {
            prev = curr;
            curr = curr->next;
        }
        if (prev) {
            prev->next = curr->next;
        }
        if (curr == rear) {
            rear = prev;
        }
        curr->next = front;
        front = curr;

    }

    // Display the queue
    void display() {
        Node* temp = front;
        while (temp) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }
};

// Main function
int main() {
    Queue q;
    q.enqueue(5);
    q.enqueue(11);
    q.enqueue(34);
    q.enqueue(67);
    q.enqueue(43);
    q.enqueue(55);

```

```
int n = 3; // Move 3rd element (34) to the front

cout << "Original queue: ";
q.display();

q.moveNthFront(n);

cout << "Modified queue: ";
q.display();

return 0;
}
```

Expected Output

If implemented correctly, the program should produce the following output:

```
Original queue: 5 11 34 67 43 55
Modified queue: 34 5 11 67 43 55
```

- (b) You are given a stack of N elements; your task is to write a function to reverse only the second half of the stack while keeping the first half in the same order. The stack follows LIFO (Last-In, First-Out) order, meaning the last inserted element is on the top. You are required to complete the missing code.

```
#include <iostream>
using namespace std;

// Stack Class
class Stack {
private:
    int* arr;
    int top;
    int capacity;

public:
    // Constructor
    Stack(int size) {
        capacity = size;
        arr = new int[capacity];
        top = -1;
    }

    // Push an element onto the stack
    void push(int value) {
```

1 mark

```
        if (top < capacity - 1)
        {
            arr[++top] = value;
        }
```

```
}
```

```
// Pop an element from the stack
```

```
1 mark int pop() {
```

```
    if (top >= 0)
    {
        return arr[top--];
    }

    return -1; // Stack underflow
```

```
}
```

```
// Get the top element of the stack
```

```
1 mark int top() {
```

```
    if (top >= 0)
    {
        return arr[top];
    }

    return -1; // Stack is empty
```

```
}
```

```
// Check if the stack is empty
```

```
bool isEmpty() {
    return top == -1;
}
```

```
// Get the size of the stack
```

```
int size() {
    return top + 1;
}
```

```
// Display stack elements (for debugging)
```

```
1 mark void display() {
```

```
    for (int i = 0; i <= top; i++)
    {
        cout << arr[i] << " ";
    }

    cout << endl;
```

```
}

// Destructor
~Stack() {
    delete[] arr;
}

};
```

```
// Function to reverse the second half of the stack
```

```
2 marks void reverseSecondHalf(Stack &s) {
```

```
    int n = s.size();
    int half = n / 2;
    Stack tempStack(half);

    // Push second half elements onto tempStack
    for (int i = 0; i < half; i++)
    {
        tempStack.push(s.pop());
    }

    // Push back reversed elements into original stack
    for (int i = 0; i < half; i++)
    {
        s.push(tempStack.pop());
    }
}
```

```
}

// Main function
int main() {
    Stack s(8);
    s.push(10);
```



```

        s.push(20);
        s.push(30);
        s.push(40);
        s.push(50);
        s.push(60);
        s.push(70);
        s.push(80);

        cout << "Original Stack (Top to Bottom): ";
        s.display();

        reverseSecondHalf(s);

        cout << "Stack after Reversing Second Half (Top to Bottom): ";
        s.display();

        return 0;
}

```

```

input  Stack = [10, 20, 30, 40, 50, 60, 70, 80]  (N = 8)
output Stack = [10, 20, 30, 40, 80, 70, 60, 50]

```

Question 12. Inheritance and Polymorphism 10 marks

You are designing a **Ride-Sharing Application** that includes different types of vehicles. The system should efficiently manage the cars and motorcycles while ensuring scalability for future vehicle types.

Scenario:

A junior developer proposes the following design:

- A base class *Vehicle* with attributes: *Brand*, *Price*, and *Fuel Type*.
- Derived classes *Car* and *Motorcycle*, each with additional attributes (*seatingCapacity* for cars and *helmetRequired* for motorcycles, etc.)
- A function *displayDetails()* in the *Vehicle* class is overridden in the derived classes to print specific details.
- A separate class *TaxCalculator* with a function *calculateTax(Vehicle v)*, which determines tax based on vehicle type:
 - Cars: 10% of price
 - Motorcycles: 5% of price

a) Identify and explain the issues

1 mark i. Identify two major design issues in the proposed structure violating OOP principles

- The *TaxCalculator* class takes a vehicle object as input but determines tax using conditional statements (if-else or switch case). Every time a new vehicle is introduced, *TaxCalculator* must be modified. This violates the Open/Closed Principle because the class should be open for extension but closed for modification.
- The *displayDetails()* function is defined in the base class and overridden in the child classes. However if a vehicle is not declared as an abstract class, there is no guarantee that all derived classes will provide their own implementation.

- 1 mark ii. Explain why they violate core OOP principles such as inheritance, polymorphism, encapsulation, or abstraction

- **Inheritance:**
Proper inheritance is used, but TaxCalculator does not utilize polymorphism, leading to tight coupling.
- **Polymorphism:**
The tax calculation should be polymorphic rather than relying on explicit type checks inside TaxCalculator.
- **Encapsulation:**
Directly passing a Vehicle object to TaxCalculator breaks encapsulation because it depends on internal properties.
- **Abstraction:**
Vehicle should be an abstract class so displayDetails() is properly implemented in derived classes.

- b) Suggest a better approach

- 1 mark i. Suggest an improved class design for handling tax calculation

Instead of a separate TaxCalculator class with conditional logic, we should use polymorphism:

- Define an abstract class Vehicle with a calculateTax() method.
- Each vehicle type (Car, Motorcycle, etc.) overrides calculateTax() to provide its own tax calculation.
- This eliminates the need for an external TaxCalculator class.

- 1 mark ii. Explain how your design ensures scalability by supporting future vehicle types

- **Scalability:**
New vehicle types (e.g., Truck, ElectricCar) can be added without modifying existing code. This way each class handles its tax calculation independently, avoiding redundancy.

- c) Role of Abstract classes, Interface, and Polymorphism

- 1 mark i. Should TaxCalculator have been an abstract class or an interface? Explain

Instead of TaxCalculator, tax calculation should be in Vehicle as a pure virtual function.

If a TaxCalculator class is still needed (e.g., for external tax policies), it can be an interface (a class with only pure virtual functions).

- 1 mark ii. Explain how polymorphism improves the ride-sharing app's ability to handle different vehicle types. Provide an example of polymorphic behavior used in real-world applications like Uber or Careem

Polymorphism allows the app to handle different vehicle types (Cars, Bikes, Rickshaws) uniformly.

Example: Uber dynamically calculates fares for different ride types (UberGo, UberX, UberMoto) based on vehicle type.

- d) Implement the chosen design

- 3 marks i. Write the code for the Vehicle class hierarchy, including Car and Motorcycle. Also include the improved alternate to TaxCalculator in this code while ensuring it properly integrates with this Vehicle class hierarchy.

```
#include <iostream>
using namespace std;

// Abstract Base Class
class Vehicle {
protected:
    string brand;
    double price;
    string fuelType;
public:
    Vehicle(string b, double p, string f) : brand(b), price(p), fuelType(f) {}
    virtual void displayDetails() = 0; // Pure virtual function
    virtual double calculateTax() = 0; // Ensures polymorphic behavior
    virtual ~Vehicle() {} // Virtual destructor
};

// Car Class (Derived)
class Car : public Vehicle {
private:
    int seatingCapacity;
public:
    Car(string b, double p, string f, int seat) : Vehicle(b, p, f), seatingCapacity(seat) {}

    void displayDetails() override {
        cout << "Car: " << brand << ", Price: " << price << ", Fuel: " << fuelType << ", Seats: " <<
            seatingCapacity << endl;
    }

    double calculateTax() override { return price * 0.10; // 10% tax
    }
};
```

```
// Motorcycle Class (Derived)
class Motorcycle : public Vehicle {
private:
    bool helmetRequired;

public:
    Motorcycle(string b, double p, string f, bool helmet) : Vehicle(b, p, f), helmetRequired(helmet) {}

    void displayDetails() override {
        cout << "Motorcycle: " << brand << ", Price: " << price << ", Fuel: " << fuelType << ", Helmet \
            Required: " << (helmetRequired ? "Yes" : "No") << endl;
    }

    double calculateTax() override {
        return price * 0.05; // 5% tax
    }
};
```

- 1 mark ii. Write a main function to demonstrate behavior where different vehicle types calculate tax dynamically. Also, write the expected output of this main function.

```
// Demonstration of Polymorphism
int main() {
    Vehicle* v1 = new Car("Toyota", 30000, "Petrol", 5);
    Vehicle* v2 = new Motorcycle("Honda", 15000, "Petrol", true);

    v1->displayDetails();
    cout << "Tax: $" << v1->calculateTax() << endl;
    v2->displayDetails();
    cout << "Tax: $" << v2->calculateTax() << endl;
    delete v1;
    delete v2;
    return 0;
}
```